

Cartivo Merchant — Technical Manual

Architecture, environment, the device-auth chain, the live-orders socket, build and deployment.

Written for

Developers, Integrators & IT Staff

Contents

1. Overview & tech stack
2. Repository layout
3. Environment (.env) reference
4. Local development & code generation
5. The device authentication chain
6. The live-orders WebSocket feed
7. REST endpoints
8. Local database & secure storage
9. State management & providers
10. Notifications
11. Android configuration
12. Building a release
13. Troubleshooting
14. Known constraints & current state

Use your PDF reader's bookmarks / outline panel to jump between sections.

NOTE The app lives in the `mobile_merchant/` folder of the repository. A Flutter SDK (Dart 3.11+), Android tooling, and a populated `.env` file are assumed.

1. Overview & tech stack

Cartivo Merchant is a single-screen Flutter Android app that authenticates a device against the Cartivo / DPO backend, opens a WebSocket to the merchant's live order-event stream, and lets an operator patch order status back over REST. There is no user auth, no catalogue, and no reporting — it is a thin real-time client.

Concern	Choice
Language / SDK	Dart <code>^3.11.5</code> , Flutter (stable)
State management	<code>flutter_riverpod 3.x</code> + <code>riverpod_annotation</code> / <code>riverpod_generator</code>
DI	<code>get_it 8</code> + <code>injectable 2</code> (<code>configureDependencies()</code> in <code>bootstrap.dart</code>)
HTTP	<code>dio 5</code> , one instance owned by <code>ApiClient</code>
Realtime	<code>web_socket_channel 3</code> (<code>IOWeSocketChannel</code>)
Routing	<code>go_router 16</code> via <code>go_router_builder</code> -style codegen (<code>appRouterProvider</code>)
Local DB	<code>drift 2.28</code> + <code>sqlite3_flutter_libs</code>
Secure storage	<code>flutter_secure_storage 9</code>
Models	<code>freezed 3</code> + <code>json_serializable 6</code> (DTOs); some hand-written DTOs where leniency is needed
Notifications	<code>flutter_local_notifications 22</code>
Device fingerprint	<code>device_info_plus 11</code> , <code>package_info_plus 8</code>
Env	<code>flutter_dotenv 5</code> — <code>.env</code> is a bundled asset
Result type	<code>Either<Failure, T></code> (<code>dartz</code>) at the repository boundary

NOTE `custom_lint` / `riverpod_lint` are intentionally omitted (incompatible plugin entrypoints on this analyzer). Re-add them together with the `custom_lint` line in `analysis_options.yaml` once a compatible pair ships.

2. Repository layout

```
mobile_merchant/
|- lib/
|  |- main.dart           # entry -> bootstrap()
|  |- bootstrap.dart      # dotenv.load -> DI -> SettingsService.init -> notifications -> runApp
|  |- app.dart            # MaterialApp.router, SessionExpiredListener shell
|  |- core/
|     |- config/          env_config.dart, app_config.dart
|     |- di/              injection.dart (+ injection.config.dart)
|     |- network/         api_client.dart, api_endpoints.dart, interceptors/
|     |- auth/            session_expired_notifier.dart, session_manager.dart
|     |- router/          app_router.dart, app_routes.dart, route_guards.dart (stub)
|     |- database/        app_database.dart, tables/, daos/
|     |- storage/         secure_storage.dart, merchant_device_storage.dart
|     |- services/        settings_service.dart, services/notifications/
|     |- notifications/   order_toast.dart
|     |- providers/       connectivity_provider.dart
|     |- theme/ utils/ widgets/ error/
|  |- shared/             app_scaffold, app_button, app_snackbar, ...
|  |- features/
|     |- dashboard/       the only screen; toolbar + orders body
|     |- merchant/        merchant profile + device registration / approval
|     |- orders/          live feed, orders list state, order cards, DTOs
|- android/               the only platform folder (no ios/ windows/ macos/ linux/)
|- assets/                app icon
|- .env / .env.sample
|- test/
```

Feature-module convention

Feature-first clean architecture. Each feature is a vertical slice with `data/` (datasources, models, repositories impl), `domain/` (pure Dart entities, repository interfaces, use cases), and `presentation/` (Riverpod providers, screens, widgets). `domain/` imports no Flutter or infra. Riverpod notifiers resolve dependencies with `getIt<T>()` at call time, never via constructor injection. Full rules in `lib/features/README.md`.

NOTE Only merchant and orders are fully built out. auth does not exist yet — `core/router/route_guards.dart`, `session_manager.dart` and `SessionExpiredListener` are stubs wired to activate once it does. The router has exactly one route: `'/'` -> `DashboardScreen`.

3. Environment (.env) reference

`.env` is loaded by `dotenv.load()` at the top of `bootstrap()` and is declared as an asset in `pubspec.yaml`. All reads go through `EnvConfig`; nothing else touches `dotenv` directly. Every getter has a safe fallback so widget tests run without a file.

Key	Used for	Fallback
<code>API_BASE_URL</code>	Base URL for every REST call. The WebSocket URL is <i>derived</i> from it (see below). May or may not include a path suffix like <code>/api/v1</code> depending on the target backend.	<code>http://10.0.2.2:3000/api/v1</code>
<code>APP_NAME</code>	<code>MaterialApp.title</code>	DPO Merchant
<code>ENABLE_LOGGING</code>	Verbose HTTP + app logging (<code>"true"</code> / <code>"false"</code>)	<code>false</code>
<code>SETTINGS_PASSWORD</code>	Password gating the (not yet surfaced) custom-endpoint settings dialog	<code>" "</code>
<code>WEBHOOK_SECRET</code>	Sent as <code>webhook_secret</code> in the <code>POST /auth/token</code> body — the shared secret that identifies your integration	<code>" "</code>
<code>CLIENT_ID</code>	Read by <code>EnvConfig.clientId</code> ; reserved for the auth handshake	<code>" "</code>
<code>CSV_EXPORT_PASSWORD</code>	Present in some <code>.env</code> files for parity with <code>mobile/</code> ; unused by this app	n/a

WebSocket URL derivation

`EnvConfig.wsBaseUrl` parses `API_BASE_URL`, swaps the scheme (`http`→`ws`, `https`→`wss`), keeps the host, drops the path, and drops the port only when it is the scheme default. `OrdersLiveFeedRepository` then appends `/ws` and a `?merchant_id=` query param.

```
API_BASE_URL=https://api.example.com/api/v1
-> REST : https://api.example.com/api/v1/...
-> WS   : wss://api.example.com/ws?merchant_id=ABC123

API_BASE_URL=http://192.168.1.34:4000
-> REST : http://192.168.1.34:4000/...
-> WS   : ws://192.168.1.34:4000/ws?merchant_id=ABC123
```

IMPORTANT New keys must be added as typed getters on `EnvConfig` with a fallback — never read `dotenv` elsewhere.

4. Local development & code generation

First run

```
cd mobile_merchant
cp .env.sample .env          # then fill in values
flutter pub get
dart run build_runner build --delete-conflicting-outputs
flutter run                  # a connected Android device / emulator
```

Codegen

Re-run `build_runner` after touching any `@freezed`, `@riverpod`, `@injectable`, `@JsonSerializable`, or `Drift`-annotated file. During active work:

```
dart run build_runner watch --delete-conflicting-outputs
```

Generated files (`*.freezed.dart`, `*.g.dart`, `injection.config.dart`, `app_database.g.dart`, `app_router.g.dart`) are committed and must never be edited by hand.

Checks

```
dart analyze                # static analysis (flutter_lints 6)
flutter test                # unit tests under test/
```

NOTE Per the project convention for `mobile_merchant/`, keep changes inside this folder and verify with `dart analyze` — do not build or run `be/` or `kiosk/` to test a change here.

Emulator networking

From the Android emulator, the host machine is `10.0.2.2` (the default fallback). A physical device needs the host's LAN IP in `API_BASE_URL` and the backend bound to `0.0.0.0`.

5. The device authentication chain

Every launch runs a three-call chain before the socket can open. `deviceStartupProvider` (a `FutureProvider`) runs it once per launch and memoizes the result; `MerchantNotifier.recheckDeviceStatus()` refreshes it on demand. All three calls are on `MerchantApi`.

1 — POST /auth/token

Body	<code>{ webhook_secret, merchant_id }</code> (<code>webhook_secret</code> from <code>EnvConfig.webhookSecret</code>)
Accepts	200
Returns	<code>{ merchant_id, merchant_name, token, exp }</code> — a short-lived webhook JWT scoped to that one merchant
Stored	<code>MerchantDeviceStorage.writeToken(token, exp, merchantId)</code> — token, expiry, and the merchant it is scoped to

`ensureWebhookToken(merchantId)` re-mints when the stored token is missing, within 1 minute of expiry, or `tokenMerchantId != merchantId` (left over from a previous merchant). This guard is the single source of truth for token freshness — callers must never use `storage.token` directly.

2 — POST /devices/register

Headers	<code>Authorization: Bearer <webhook JWT></code> , <code>Idempotency-Key: <install_id></code>
Body	<code>RegisterDeviceRequest: platform, install_id, name, app_version, platform_version, device_model, platform_details (android_id, manufacturer, brand, device, sdk_int, is_physical_device)</code>
Accepts	200 (idempotent replay) or 202 (fresh enrolment)
Returns	<code>{ device_id, status, device_secret?, merchant_id?, merchant_name?, requested_at?, reviewed_at?, review_note? }</code>
<code>device_secret</code>	returned only on 202 , once. Persisted then; never overwritten with null on a later 200.

`install_id` is a v4 UUID generated once (`ensureInstallId()`) and **never cleared**, even on a merchant switch — it identifies the install, not the merchant link. `status` is one of `pending` / `approved` / `deactivated` (`rejected` is bucketed as `deactivated` ; anything else is `unknown`). It is cached in secure storage (`merchant_device_status`) as an optimistic gate read on the next cold start.

3 — POST /devices/token

Headers	<code>Authorization: Bearer <webhook JWT></code>
Body	<code>{ device_id, device_secret }</code>
Accepts	<code>200</code>
Returns	<code>{ device_id, merchant_id, token, exp }</code> — a short-lived JWT used as the WS handshake bearer
Caching	Not cached across launches. <code>DeviceTokenRepository.mint()</code> is called on every connect attempt. The value is still written to storage (<code>merchant_device_ws_token*</code>) so <code>invalidate()</code> can clear it on a 401/403 handshake rejection.

Approval polling

- `DashboardScreen` polls `recheckDeviceStatus()` every **15 s** while the resolved status is non-approved and not `tokenUnavailable` .
- It also re-probes on every `AppLifecycleState.resumed` .
- On the `approved` transition the poll stops, any open status dialog is popped, and `OrdersFeedNotifier` — which watches `deviceStartupProvider` — reconnects on its own. No restart.

6. The live-orders WebSocket feed

`OrdersFeedNotifier` (an `AsyncNotifier<OrdersFeedState>`, kept alive with `ref.keepAlive()`) owns the socket lifecycle. `OrdersLiveFeedRepository` is the transport.

Handshake

```
GET {wsBaseUrl}/ws?merchant_id={merchantId}
Authorization: Bearer {device token from POST /devices/token}
pingInterval: 20s
```

Connect order mirrors `mobile/` : `ensureWebhookToken` -> `mint /devices/token` -> `IOWebSocketChannel.connect` -> await `channel.ready` with a **10 s** timeout.

Connection state machine

State	Meaning
<code>connecting</code>	Initial attempt in flight
<code>connected</code>	<code>ready</code> resolved, subscription live
<code>reconnecting</code>	Dropped or failed; retry scheduled
<code>disconnected</code>	No merchant / not approved / deactivated — nothing to connect to

Reconnect / resilience

- Exponential backoff **1 s** -> **30 s** (* 2 each try). Reset to 1 s after a connection that stayed up > 5 s.
- A `_connectGeneration` counter supersedes stale in-flight attempts so a slow handshake can't clobber a newer one.
- `connectivityStatusProvider` transition offline->online triggers an immediate reconnect when the feed is reconnecting/disconnected.
- 401/403-looking errors call `DeviceTokenRepository.invalidate()` so the next attempt re-mints.
- De-dupe: a 200-entry ring of `event_id` s survives reconnects (the upstream replays a backlog on every resubscribe); only a full `_teardown` (dispose / merchant change) clears it.
- Hydration mute: for **4 s** after the first successful connect, toasts and OS notifications are suppressed to kill the backlog-replay notification storm.

Wire frame

The WS envelope is `{ event_id, event_type, data }` — no `id` / `received_at` / `created_at` wrapper (those are synthesized). `OrderEventDto.fromWireJson` returns `null` (never throws) for: a non-string frame, non-object JSON, an `event_type` not starting `order.`, a missing `event_id`, or a non-object `data`. `OrderDataDto.fromWireJson` requires

only `id` ; every other field is lenient with defaults (the feed mixes full order events with identity-only `order.deleted` tombstones).

event_type	Handling
<code>order.created</code>	Upsert row, prepend to list, notify
<code>order.updated</code>	Upsert row, replace list entry by <code>data.id</code> , notify
<code>order.cancelled</code>	Same as updated; status -> <code>cancelled</code>
<code>order.deleted</code>	Delete local row + line items, remove from list

7. REST endpoints

Two calls, both authenticated with the webhook JWT (not the device token). Paths are relative to `API_BASE_URL` and listed in `ApiEndpoints`.

GET /merchant/orders

Query	<code>merchant_id</code>
Header	<code>Authorization: Bearer <webhook JWT></code>
Returns	<code>MerchantOrdersDto</code> — a paginated order-event stream. <code>result.events</code> is already current-state: <code>order.deleted</code> tombstones are applied and unparseable events dropped.
On success	<code>OrderEventsDao.replaceAll(merchantId, events)</code> then <code>OrdersState(events)</code>
On failure	fall back to <code>dao.getEvents(merchantId)</code> ; if that is also empty, rethrow (drives the “Couldn't load orders” state); otherwise <code>OrdersState(events: cached, isStale: true)</code> -> the red cached-data banner

PATCH /merchant/orders/{orderId}

Body	<code>{ "updates": { "status": "<status>" } }</code> — <code>cancelled</code> included
Header	<code>Authorization: Bearer <webhook JWT></code>
Accepts	<code>200</code>
Returns	<code>OrderStatusUpdateResultDto</code> — echoes the canonical <code>order.updated</code> event the backend also broadcasts on the feed, so the caller converges the list on it
Errors	<code>DioException</code> is caught and rethrown as a <code>MerchantApiException</code> with a user-facing message (<code>>=500</code> vs. connection error)

`OrdersNotifier` keeps a 200-entry `_appliedEventIds` ring so the socket echo of a change it just made via PATCH is not folded in a second time (which, with an out-of-order echo, could briefly revert the status).

Auth model summary

Token	Minted by	Used on	Lifetime
Webhook JWT	POST /auth/token	All REST calls (/devices/register , /devices/token , /merchant/orders GET + PATCH)	Short; re-minted <1 min before exp or on merchant mismatch
Device JWT	POST /devices/token	WS handshake Authorization header only	Short; re-minted every connect attempt
User JWT	POST /auth/login (not wired)	Attached by AuthInterceptor if present; a stray 401 signals session-expiry	n/a — no login screen yet

8. Local database & secure storage

Drift database

`AppDatabase` (`@DriftDatabase`), one SQLite file `mobile_merchant.sqlite` in the app documents directory, opened in a background isolate. `schemaVersion = 4` with a stepwise `onUpgrade` .

Table	Holds
<code>merchant_table</code>	The single merchant row (<code>id</code> , <code>merchant_id</code> , <code>merchant_name</code>). Added in v2.
<code>order_events_table</code>	Cached order events — plus <code>fulfillment_type</code> , <code>facility_name</code> , <code>district_name</code> added in v4. Added in v3.
<code>order_items_table</code>	Line items per cached order. Added in v3.

Accessed only through `MerchantDao` / `OrderEventsDao` from the `data/` layer — never from `domain/` . Adding a table: add the `Table` class, list it in `@DriftDatabase` , bump `schemaVersion` , add an `onUpgrade` branch, re-run codegen.

Secure storage — MerchantDeviceStorage

A dedicated wrapper over `flutter_secure_storage` , separate from the user-auth `SecureStorage` . Keys:

Key	Value
<code>merchant_webhook_token</code> / <code>_exp</code> / <code>_merchant_id</code>	The <code>/auth/token</code> JWT, its unix expiry, and the merchant it is scoped to
<code>merchant_device_id</code>	<code>device_id</code> from <code>/devices/register</code>
<code>merchant_device_secret</code>	<code>device_secret</code> (written once, on the 202)
<code>merchant_install_id</code>	v4 UUID, generated once, never cleared
<code>merchant_registered_merchant_id</code>	Which merchant this device is enrolled under
<code>merchant_device_status</code>	Last seen enrolment status (optimistic gate)
<code>merchant_device_ws_token</code> / <code>_exp</code> / <code>_merchant_id</code>	The <code>/devices/token</code> JWT (cleared on handshake rejection)
<code>custom_api_base_url</code>	Runtime endpoint override (<code>AppConfig.customApiBaseUrlKey</code>)

`clearForMerchantChange()` wipes everything above **except** `merchant_install_id` . Called from `MerchantNotifier.save()` when the Merchant ID changes, followed by a fresh `activateDevice()` .

9. State management & providers

Provider	Type	Responsibility
<code>merchantProvider</code>	<code>AsyncNotifier<Merchant?></code>	Loads / creates / updates the single merchant row. <code>register()</code> , <code>save()</code> , <code>recheckDeviceStatus()</code> , <code>syncOnStartup()</code> .
<code>deviceStartupProvider</code>	<code>FutureProvider<DeviceStartupResult></code>	Runs the launch auth chain once and memoizes. Awaited by the dashboard and by <code>OrdersFeedNotifier.build()</code> so the feed can't mint a device token before registration resolves. Invalidated explicitly by <code>MerchantNotifier.save()</code> .
<code>ordersFeedNotifierProvider</code>	<code>AsyncNotifier<OrdersFeedState></code>	Socket lifecycle + connection state. <code>keepAlive</code> . <code>checkConnection()</code> is the idempotent re-probe called from the dashboard post-frame callback and after each approval re-check.
<code>ordersProvider</code>	<code>AsyncNotifier<OrdersState></code>	The order <i>list</i> : initial REST fetch + cache fallback, <code>applyLiveEvent()</code> , <code>updateStatus()</code> / <code>cancelOrder()</code> , <code>refresh()</code> .
<code>connectivityStatusProvider</code>	<code>Stream/AsyncValue<bool></code>	Wraps <code>connectivity_plus</code> ; drives socket reconnect on the offline->online edge.

List-merge rules (subtle, easy to regress)

- `orders_body` de-dupes by `data.id` keeping the highest synthetic `id` , then sorts desc.
- `mergeLiveOrderEvent` drops any existing entry for the same `data.id` and prepends the incoming event.
- `OrdersNotifier._withStatus` rebuilds an `OrderEventDto` locally only when PATCH succeeds but echoes no usable event.
- `MerchantNotifier.syncOnStartup()` only re-emits `merchantProvider` when the row actually changed — a blind reassignment double-flashes the dashboard spinner.

10. Notifications

`OrderNotificationsService` (`flutter_local_notifications`). `initialize()` runs in `bootstrap()` before `runApp` : it sets up the Android channel `live_orders` ("Live orders", high importance) and calls `requestNotificationsPermission()` (Android 13+ runtime prompt). Every call is wrapped in a bare `try/catch` — a notification failure must never affect the feed.

`orderNotificationText(event)` is the single source of copy, shared by the OS notification and the in-app toast (`showOrderToast` via the global `appScaffoldMessengerKey`) so the two never drift. Notification id is `event.eventId.hashCode` .

event_type	Title	Body
<code>order.created</code>	New order #id	"N items · CUR total"
<code>order.cancelled</code>	Order #id cancelled	(empty)
<code>order.deleted</code>	Order #id removed	(empty)
other	Order #id updated	"Status: ..."

NOTE Both toast and OS notification are gated behind the 4 s hydration-mute window after first connect (see the feed section).

11. Android configuration

IMPORTANT THE SINGLE MOST IMPORTANT DEPLOYMENT FACT. `android/app/src/main/AndroidManifest.xml` declares only `POST_NOTIFICATIONS`. The `INTERNET` permission is present **ONLY** in the `src/debug/AndroidManifest.xml` (Flutter adds it there for the dev tooling). A **RELEASE** build therefore ships with **NO INTERNET PERMISSION** — and this app is useless without the network. You must add `<uses-permission android:name="android.permission.INTERNET"/>` to the **MAIN** manifest before shipping.

Manifest facts

Item	Value
<code>android:label</code>	<code>Cartivo Merchant</code>
Launch activity	<code>.MainActivity</code> , <code>singleTop</code> , <code>exported</code>
Permissions (main)	<code>POST_NOTIFICATIONS</code> only
Permissions (debug overlay)	<code>INTERNET</code>
Icon	<code>@mipmap/ic_launcher</code> (<code>flutter_launcher_icons</code> , source <code>assets/icon/app_icon.png</code> , <code>min_sdk_android: 21</code>)

Recommended main-manifest additions before release

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

`ACCESS_NETWORK_STATE` is pulled in transitively by `connectivity_plus` but is worth making explicit.

Platform coverage

Android only. There is no `ios/` , `windows/` , `macos/` , or `linux/` folder. `flutter_launcher_icons` is configured with `ios: false` . The notification service references `DarwinInitializationSettings` for future parity only.

12. Building a release

APK / App Bundle

```
cd mobile_merchant
cp .env.sample .env          # production values – real API_BASE_URL,
                              # WEBHOOK_SECRET, CLIENT_ID

flutter pub get
dart run build_runner build --delete-conflicting-outputs
flutter build apk --release    # or: flutter build appbundle --release
```

Pre-flight checklist

- `INTERNET` permission added to the **main** manifest (see previous section).
- `.env` holds the production `API_BASE_URL` (with `https://` so the socket upgrades to `wss`).
- `WEBHOOK_SECRET` and `CLIENT_ID` are the real integration credentials, not the sample values.
- `ENABLE_LOGGING=false` for production.
- App signing configured in `android/app/build.gradle(.kts)` (`flutter build` uses the debug key otherwise).
- `version:` in `pubspec.yaml` bumped (currently `1.0.0+1`).
- Test on a **physical device** with a real (non-emulator) backend URL — the emulator's `10.0.2.2` fallback masks the `INTERNET`-permission bug.

IMPORTANT `.env` is bundled into the APK as an asset. Anything in it — including `WEBHOOK_SECRET` — is extractable from the built APK. Treat the webhook secret as a low-trust integration key, scope it tightly server-side, and rotate it if an APK leaks.

13. Troubleshooting

Socket never reaches “connected”

- **Release build, no orders, pill stuck:** missing `INTERNET` permission. #1 cause. Check the merged manifest in the built APK.
- **disconnected not reconnecting**: the guard in `build()` / `checkConnection()` found no merchant, no webhook token, or a cached non-approved status. Not a network problem — check `deviceStartupProvider`.
- **Handshake 401/403:** device token rejected — check `device_secret` is present, the webhook JWT is for the right merchant, and the device is `approved` server-side.
- **ready times out (10 s):** wrong `wsBaseUrl` (check the `[OrdersFeed]` handshake – `uri:` debug line), or the backend has no `/ws` route on that host.

Enable verbose logging

Set `ENABLE_LOGGING=true` in `.env`. `LoggingInterceptor` then logs every request/response; `OrdersFeedNotifier` and `OrdersLiveFeedRepository` emit `[OrdersFeed]` -prefixed `debugPrint`s (handshake URI, decoded JWT claims, close code/reason, uptime on drop). Filter `flutter logs` / `logcat` on `OrdersFeed`.

Orders load but status changes fail

PATCH uses the webhook JWT via `ensureWebhookToken(merchant.merchantId)`. A 403 “merchant_id does not match” means a stale token scoped to a previous merchant — should self-heal on the next call; if not, clear storage and re-register.

Reset a device completely

1. Android **Settings** > **Apps** > **Cartivo Merchant** > **Storage** > **Clear storage** (wipes secure storage + the SQLite file, including `install_id`).
2. Re-open the app; the **Register Merchant** dialog returns.
3. Re-register — the backend sees a new `install_id`, so this enrolls as a brand-new device needing fresh approval.

Network retry behaviour

`RetryInterceptor` retries **only** pure connection errors / connection timeouts, up to `AppConfig.maxNetworkRetries` (2). It never retries once the server has responded. Timeouts: `connectTimeout 10 s`, `receiveTimeout 30 s`. `validateStatus` treats anything `< 500` as a non-throwing response — `MerchantApi._assertSuccess` does the real status check.

14. Known constraints & current state

- **Android only.** No other platform folders.
- **No user auth.** No login screen, no PIN, no per-user audit. The `auth` feature, route guards, `SessionManager` and `SessionExpiredListener` are stubs.
- **One route.** `'/' -> DashboardScreen` . `AppRoutes.login` is reserved but unwired.
- **Custom-endpoint UI is not surfaced.** `SettingsService` + `SETTINGS_PASSWORD` support a runtime `API_BASE_URL` override (`custom_api_base_url` in secure storage), but the toolbar gear opens `MerchantFormDialog` , not an endpoint dialog. The override can only be set programmatically / from a future screen.
- **No offline write queue.** Status changes and cancellations require a live connection; a failed change is simply lost and must be redone.
- **Single merchant per device.** Switching Merchant ID is a destructive re-enrolment.
- **device_secret is single-issue.** If secure storage is cleared without a matching server-side device delete, the device row is orphaned — re-register produces a new `install_id` and a new device.
- **Currency symbol is hard-coded** to ₪ in the order card / sheet, regardless of the order's actual `currency` field (which *is* used in notification text).

Parity note

Much of this app is a deliberate port of the live-orders subsystem in the sibling `mobile/` POS app — the device-status buckets and copy (`DeviceStatusVisual`), the connect-chain order, the de-dupe ring, and the hydration mute all mirror it. When changing behaviour here, check whether `mobile/` needs the same change.